Design and Implementation of a Cloud-Native Reactive Architecture

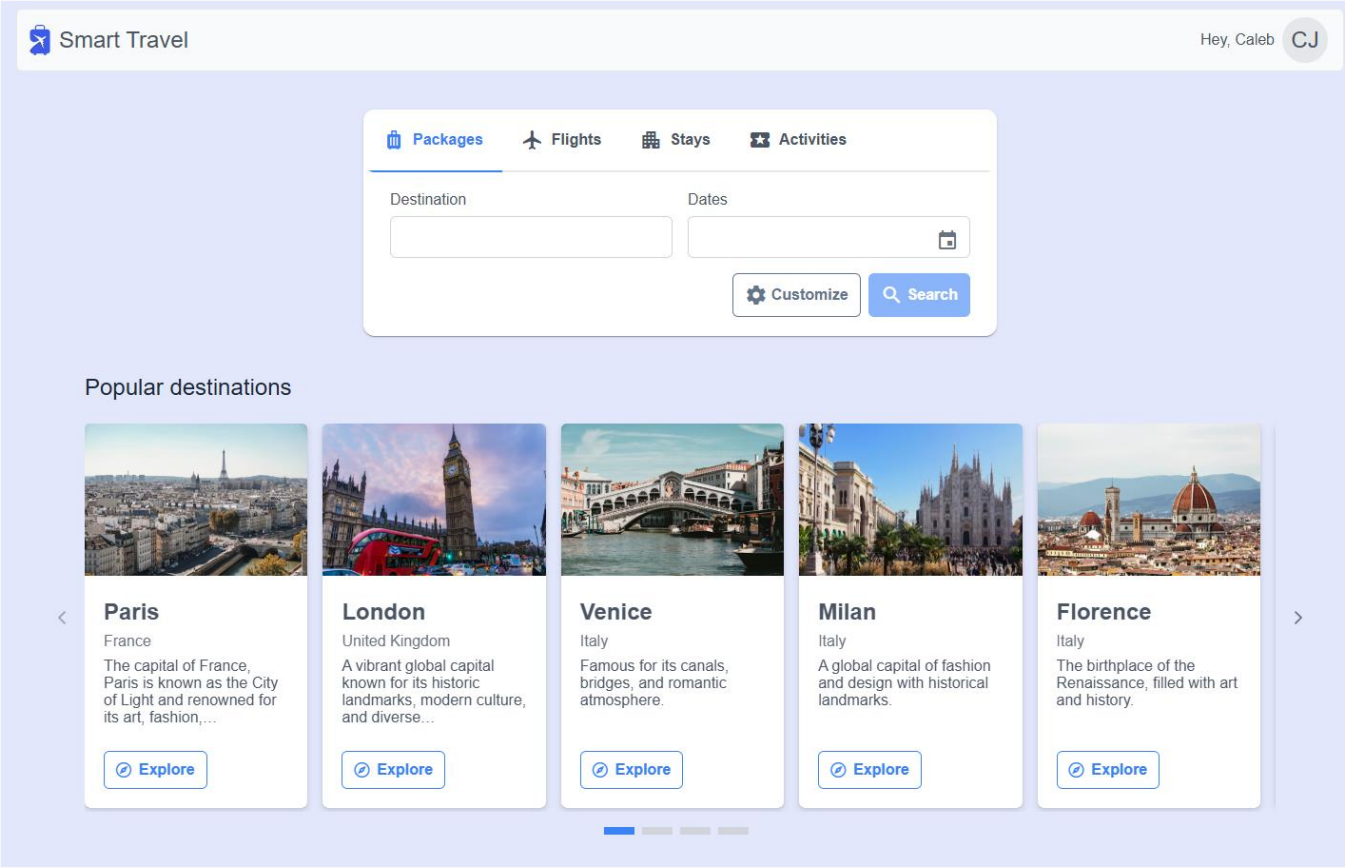
The Smart Travel Platform Case Study

Candidate: Federico Palazzi
Supervisor: *Prof*. Giovanni Malnati

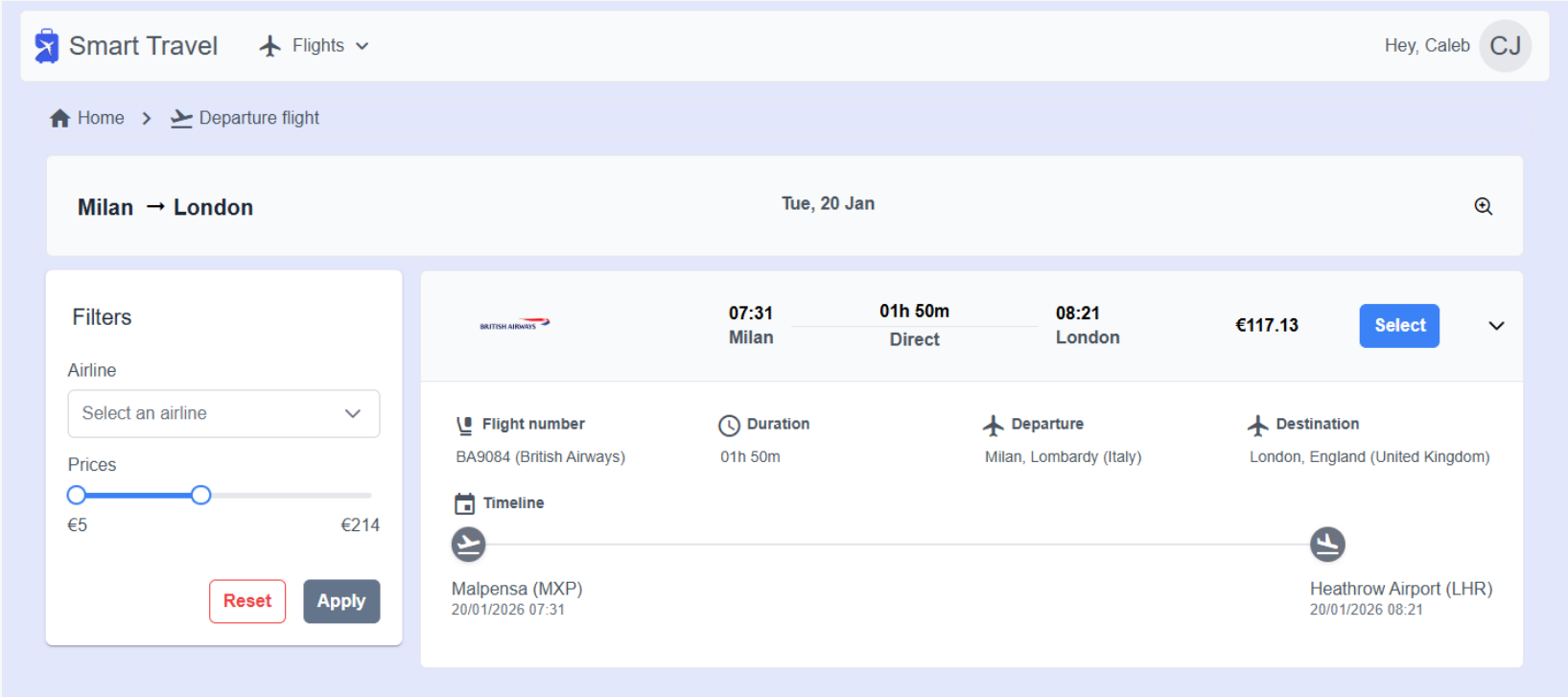
Requirements and User Roles

- Full-stack **e-commerce web application** for purchasing **travel packages** (flights, hotels, activities).
- **Requirements**
  - Public access to the travel catalog and search for individual components.
  - Purchase management and order status tracking.
  - Integration with external services (e.g., online payment gateways).
  - Email notifications for purchase confirmations.
- **User Roles**
  - CUSTOMER – Browse catalog and buy travel packages.
  - TRAVEL AGENT – Create, publish and archive packages.
  - ADMIN – Manage user accounts.

Smart Travel – Homepage



Smart Travel – Flight Search



Smart Travel – Checkout

Smart Travel

Hey, Caleb CJ

Review your choices

You can still change your selections

Departure flight

Date: 20/01/2026 **Quantity:** 2

BRITISH AIRWAYS	07:31 Milan	01h 50m Direct	08:21 London	€117.13	

Return flight

Date: 30/01/2026 **Quantity:** 2

BRITISH AIRWAYS	18:28 London	01h 50m Direct	21:18 Milan	€76.07	

Cost overview

Departure flight	€117.13 x 2 people
Return flight	€76.07 x 2 people
Total	**€386.40**

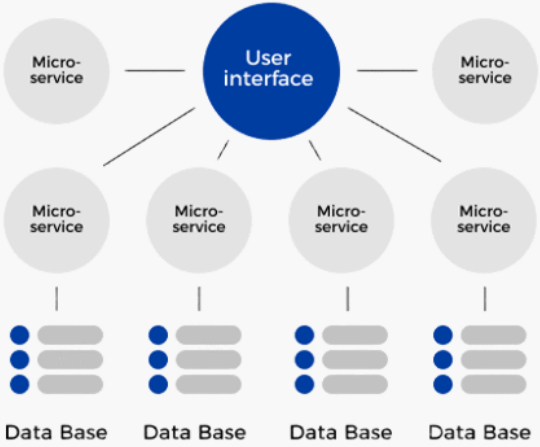
Cancel order

Checkout order

Why Cloud-Native? From Monoliths to Microservices

MONOLITHIC ARCHITECTURE

MICROSERVICE ARCHITECTURE

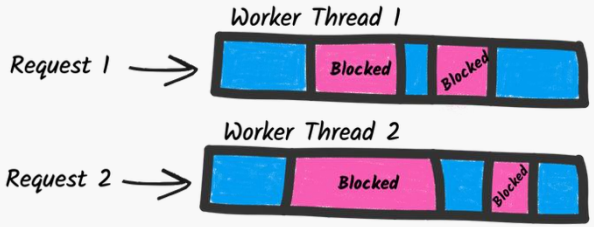


- ❌ Single point of failure
- ❌ Hard to scale under heavy traffic
- ❌ Slow release cycles
- ✅ Simpler to build and deploy

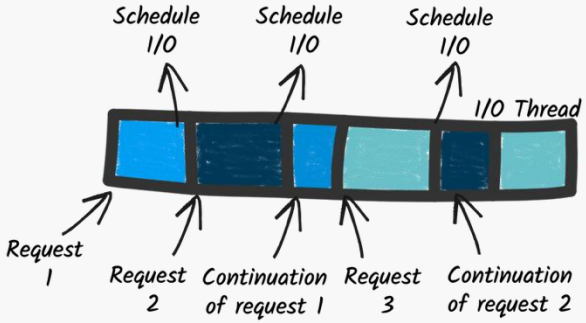
- ✅ Fault isolation (If one fails, the rest survive)
- ✅ Scale only what is needed
- ✅ Independent deployments
- ❌ High operational complexity

Why Reactive? From Imperative to Reactive Model

IMPERATIVE MODEL



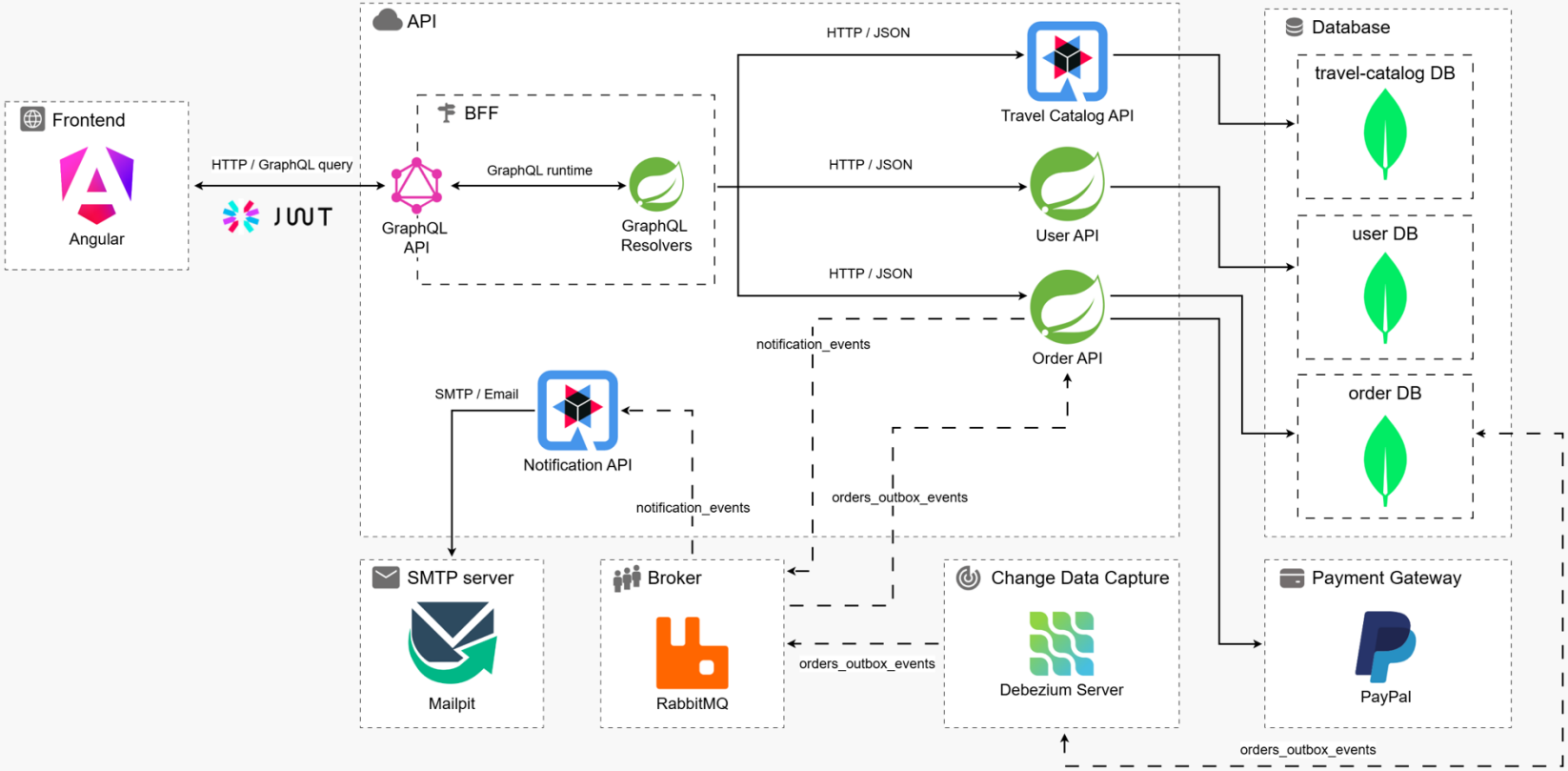
REACTIVE MODEL



- ❌ One dedicated thread per request (**worker thread**)
- ❌ Blocking I/O operations (thread waits for response)
- ❌ Limited scalability (higher resources usage)
- ✅ Simpler programming model (sequential code)

- ✅ One thread can handle multiple requests (**I/O thread**)
- ✅ Non-blocking I/O operations (thread is released)
- ✅ High concurrency (reduced resources usage)
- ❌ Programming based on continuations (callbacks)

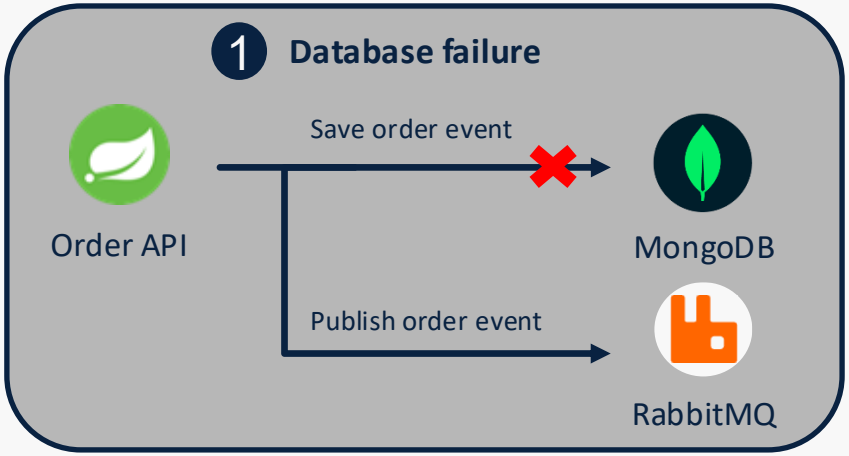
System Architecture

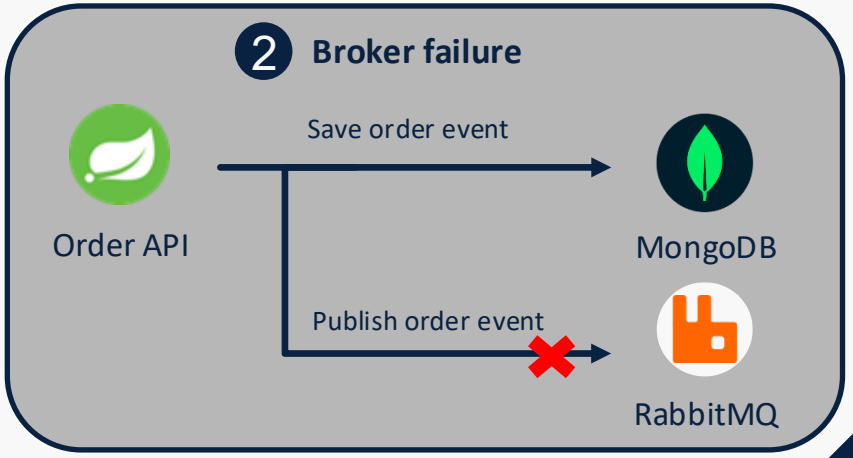


Dual-Write problem and Outbox Pattern

"The dual-write problem occurs when two external systems must be updated in an atomic fashion."

Without Outbox Pattern

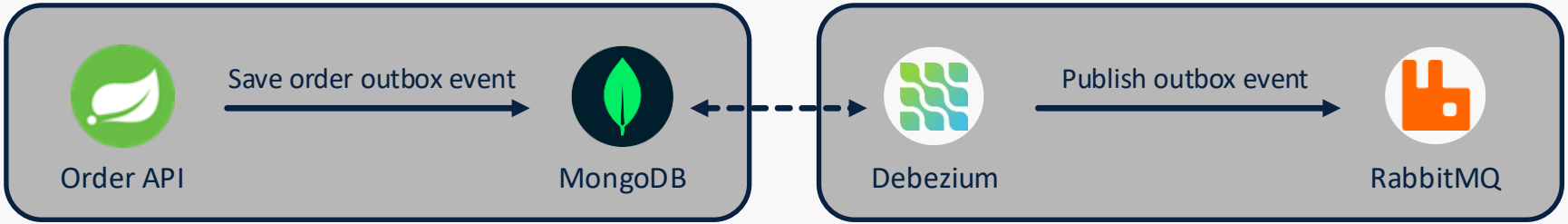




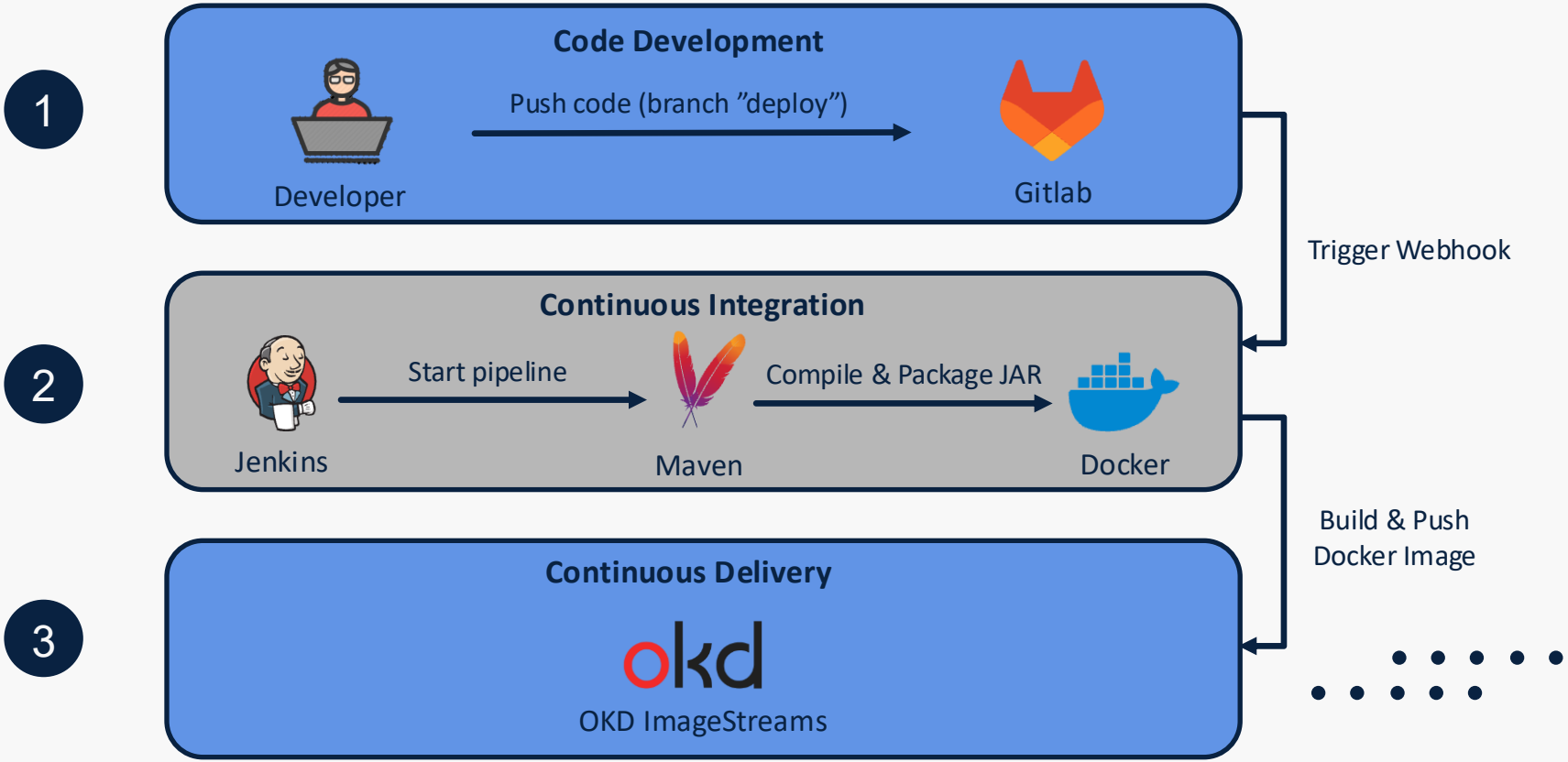
Dual-Write problem and Outbox Pattern

“The dual-write problem occurs when two external systems must be updated in an atomic fashion.”

With Outbox Pattern



Continuous Integration / Continuous Delivery



OKD – ImageStreams View

okd

3 fpalazzi

Administrator

Home

Operators

Workloads

Networking

Storage

Builds

BuildConfigs

Builds

ImageStreams

Observe

Compute

Project: stage-palazzi

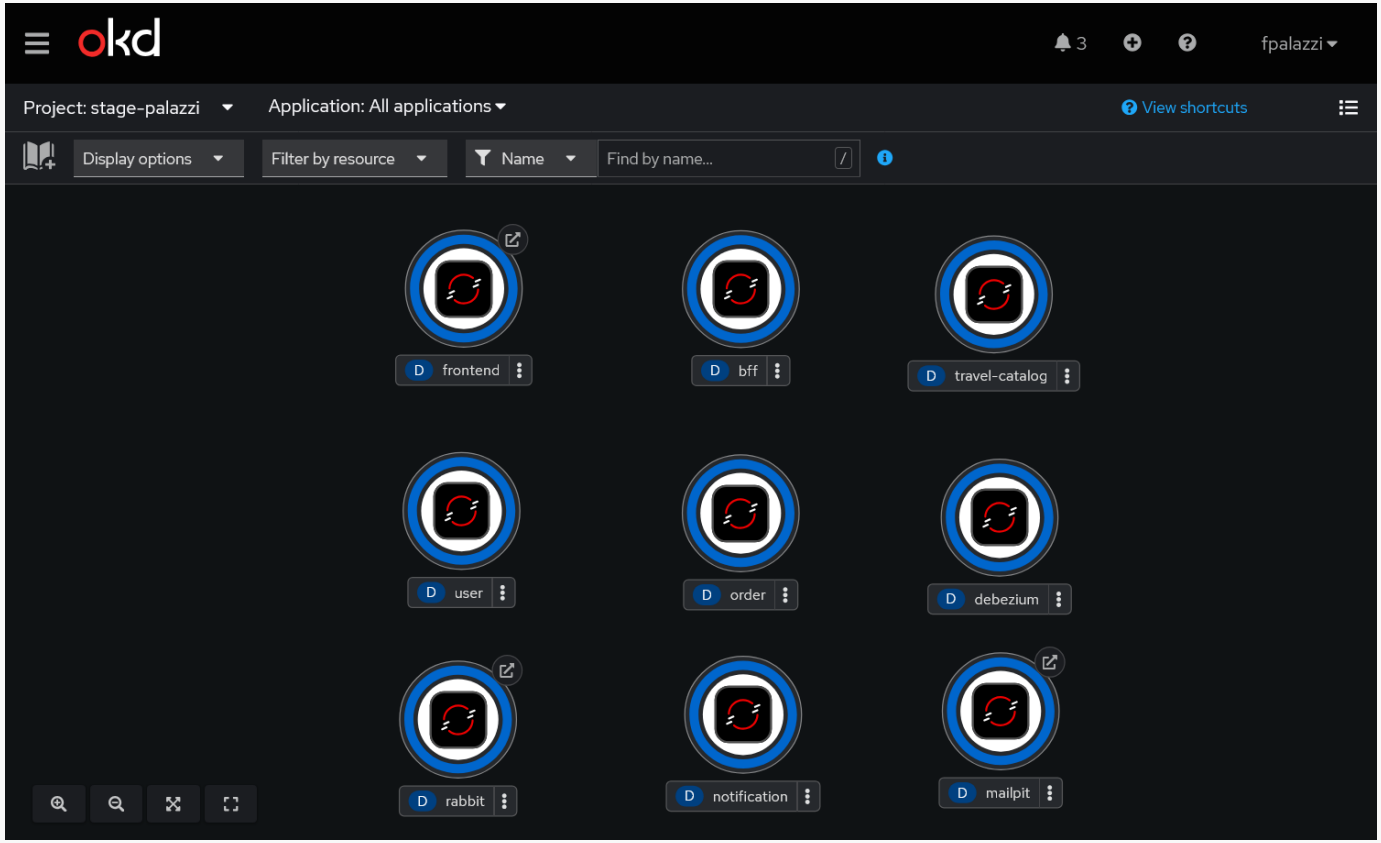
ImageStreams

Create ImageStream

Name Search by name... /

Name	Labels	Created
bff-api	No labels	8 ago 2025, 11:51
debezium	No labels	4 set 2025, 17:05
frontend	No labels	4 set 2025, 11:04
notification-api	No labels	5 set 2025, 10:41
orders-api	No labels	5 set 2025, 10:41
rabbit	No labels	4 set 2025, 17:05
travel-catalog-api	No labels	8 ago 2025, 14:10
user-api	No labels	4 set 2025, 15:17

OKD – Topology View



Conclusions – What did I learn?

- **Reactive Microservice Development**: Designed and implemented non-blocking backend services using Spring WebFlux and Quarkus Mutiny to handle high concurrency with minimal resource footprint.
- **API Gateway & Security**: Built a GraphQL Backend-For-Frontend (BFF) to optimize client data fetching, secured by stateless JWT authentication and role-based access control.
- **Distributed Data Consistency**: Solved the dual-write problem by implementing the Outbox Pattern, integrating Debezium (CDC) and RabbitMQ for reliable, asynchronous event streaming.
- **Cloud Deployment & DevOps**: Created automated CI/CD pipelines using Jenkins and deployed the containerized microservice infrastructure onto an OKD (Kubernetes) cluster.

Thank you for your attention. Are there any questions?

Technologies and Tools

Frontend

- Angular
- PrimeNG

Backend

- Quarkus
- Spring Boot

Event Handling

- Debezium
- RabbitMQ

Database

- MongoDB
- MongoDB Compass

API Design & Testing

- GraphQL
- Postman

DevOps

- Jenkins
- OKD